

1.1 Project Description

My project is a baking assistant that uses an AI agent to help people find and prepare recipes. A user can describe what they want to bake, and the system searches for recipes that fit their available time, dietary restrictions, ingredients, and taste. It also checks which ingredients the user is missing, scales recipes to a number of servings, makes shopping lists, and saves favourite recipes. The user can do all of this through the GUI or through the command line.

The problem it addresses is that it is hard for home bakers to find a recipe that matches what they have and what they need. Baking is also easy to get wrong, and when an ingredient is missing or something goes wrong in the middle of a recipe, most people have to search online for answers. The system is meant for home bakers, both beginners who need help and more experienced people who want to adjust recipes.

The agent handles the parts that need language understanding. It reads the user's request, searches for and recommends recipes, suggests substitutions for missing ingredients, changes a recipe to match a taste preference, and helps troubleshoot problems. The parts with exact answers, such as comparing ingredients, scaling quantities, building shopping lists, and storing favourites, are done by regular code. This keeps them fast and consistent, and it means the model can't make mistakes on simple calculations.

An agent suits this problem because baking requests are written in many different ways with various requirements. A set of menus and filters cannot handle that well. The agent decides what the user is asking for, picks the right tool, and can do more than one step, such as searching for recipes and then ranking them by time. If a request is unclear, it asks the user a question first.

The system uses Google Gemini through the Gemini API, with a smaller and faster model since the requests are short. Gemini was chosen because it has a free tier. The rest of the system does not depend on it directly. It is accessed through an LLMProvider interface.

A request goes from the GUI or CLI to AgentController, which passes it to BakingAgent. The agent uses LLMClient to work out what the user wants, then gets the matching tool from ToolManager, such as RecipeSearchTool or SubstitutionTool. Each tool does its job, and if it needs the model, it calls LLMClient. For exact operations it uses regular classes such as IngredientManager and Recipe. LLMClient talks to the external model through GeminiAdapter, which implements LLMProvider. The result is passed back along the same path and shown to the user.

1.2 Feature Specification

F01 - Baking Request

The user describes what they want to bake through the GUI, including preferences such as quantity, ingredients, or dietary restrictions. The feature is AI-based. It analyzes the request, identifies the user's requirements, and uses them to begin the baking process. If the request is unclear, the agent asks the user for clarification.

F02 - Recipe Search

The user enters a baking request or search criteria through the GUI. The agent searches the recipe database and retrieves recipes that match the user's requirements. This feature is hybrid, as AI interprets the user's request while the system performs the recipe search. If no suitable recipes are found, the user can modify their search.

F03 - Time Based Recipe Recommendation

The user enters how much time they have available through the GUI. The agent analyzes the available time and recommends recipes that can realistically be completed within that time. This feature is hybrid, as the system filters recipes based on their preparation and baking times while the AI ranks and explains the recommendations. If no recipes fit the available time, the agent suggests recipes requiring less time or asks the user to increase their available time.

F04 - Ingredient Availability Check

The user enters the ingredients they currently have and selects a recipe. The system compares the available ingredients with the recipe requirements and identifies any missing ingredients. This feature is deterministic because it uses predefined comparisons rather than AI.

F05 - Ingredient Substitution

The user selects a missing ingredient and asks the agent for a substitute. This feature is AI based. The recipe is analyzed and a suitable replacement is given while explaining how it may affect the recipe. If no suitable substitute is available, an alternative recipe is suggested.

F06 - Recipe Scaling

The user enters the desired number of servings or baked items for a selected recipe. The system calculates the required ingredient quantities and displays the updated recipe. This feature is deterministic because the quantities are calculated using predefined scaling rules.

F07 - Taste Customization

The user asks the agent to modify a selected recipe based on a taste preference, such as making it less sweet. This feature is AI-based. It analyzes the request and adjusts the recipe while maintaining its overall structure. The modified recipe is displayed for the user.

F08 - Shopping List Generation

The user selects a recipe and asks the system to create a shopping list. The system compares the required ingredients with the user's available ingredients and generates a list of missing items. This feature is deterministic because it uses predefined ingredient comparisons.

F09 - Recipe Favorites

The user can save recipes they like by selecting a favorite option in the GUI. The system stores the selected recipes and allows the user to view them later. This feature is deterministic because it only involves storing and retrieving recipes. If a recipe is already saved, the system prevents it from being added twice.

F10 - Baking Troubleshooter

The user describes a problem they are experiencing while preparing a recipe. The agent analyzes the problem and the current recipe step to suggest possible causes and solutions. This feature is AI-based because the agent interprets the user's description and reasons about possible solutions. If more information is needed, the agent asks the user for additional details.

2.1 Class Diagram Design Patterns

MVC: The MVC pattern separates the user interface from the application's main logic. The BakingGUI and BakingCLI classes act as the Views, while AgentController acts as the Controller and coordinates requests. Classes such as BakingRequest, Recipe, Ingredient, and ShoppingList represent the Model. This pattern is appropriate because the system has both a GUI and CLI that need to use the same functionality. Without MVC, the interface classes would contain more application logic, leading to duplicated code and making the system harder to maintain.

Strategy: The Strategy pattern allows the baking agent to select different tools depending on the user's request. AgentTool is the Strategy interface, while BakingRequestTool, RecipeSearchTool, TimeRecommendationTool, SubstitutionTool, TasteCustomizationTool, and TroubleshootingTool are concrete strategies. BakingAgent selects the appropriate tool and ToolManager manages the available tools. This pattern is appropriate because each baking task has different processing logic but follows the same tool structure. Without Strategy, the agent would require many conditional statements to handle different tasks, making it harder to add new capabilities.

Factory Method: The Factory Method pattern centralizes the creation of the different agent tools. ToolFactory is responsible for creating objects that implement the AgentTool interface, such as RecipeSearchTool, SubstitutionTool, and TroubleshootingTool. This is appropriate because the system has multiple types of tools that may need to be created depending on the user's request. Without the pattern, tool creation would be spread across classes such as BakingAgent and ToolManager, making changes to tool creation more difficult.

Facade: The Facade pattern provides a simple interface to the more complex agent subsystem. AgentController acts as the Facade, while BakingGUI and BakingCLI act as clients. Internally, the controller works with BakingAgent, ToolManager, AgentTool, and LLMClient. This pattern is appropriate because the user interfaces should not need to know how the AI agent processes requests internally. Without the Facade, the GUI and CLI would need to directly interact with multiple internal components, increasing dependencies and making the system more difficult to change.

Adapter: The Adapter pattern allows the system to communicate with an external LLM without making the rest of the application dependent on its specific API. LLMProvider defines the interface expected by the application, while OpenAIAdapter translates requests between LLMClient and the external LLM service. This pattern is appropriate because the LLM provider could be changed in the future without redesigning the rest of the system. Without the Adapter, classes such as LLMClient and the baking tools could become tightly coupled to one specific LLM provider, making future changes more difficult.

2.2 Use-Case Descriptions

UC01: Submit Baking Request

Use Case ID	UC01
Use Case Name	Submit Baking Request
Actor(s)	User, LLM Service
Goal	Describe what the user wants to bake so the system understands the requirements.
Preconditions	The application is running and the LLM Service is reachable.
Trigger	The user types a baking request and clicks Submit.
Main Success Scenario	1. The user enters the request in the GUI. 2. The system sends it to the agent. 3. The agent uses the LLM to extract requirements 4. The system displays the interpreted requirements. 5. The system performs UC02 Search Recipes («include»). 6. The matching recipes are displayed.
Alternative / Exception Flows	1. Input is empty: the system asks the user to enter a request. 2. Request is unclear: the agent asks a clarification question, the user answers, and the flow resumes at step 3. 3. LLM Service is unavailable: the system shows an error and lets the user retry.
Postconditions	A BakingRequest has been created and matching recipes are displayed.
Related Feature(s)	F01

UC02: Search Recipes

Use Case ID	UC02
Use Case Name	Search Recipes
Actor(s)	User, LLM Service
Goal	Find recipes that match the user's requirements.
Preconditions	The recipe database is available and search criteria exist.

Trigger	The user submits search criteria, or UC01 or UC03 invokes this use case.
Main Success Scenario	1. The criteria are provided. 2. The agent uses the LLM to interpret them. 3. The system searches the recipe database. 4. The matching recipes are displayed. 5. The user selects a recipe to view.
Alternative / Exception Flows	3. No recipes match: the system tells the user and invites them to modify the search. 2. LLM Service is unavailable: the system shows an error and lets the user retry.
Postconditions	A list of matching recipes is displayed, and a selected recipe becomes the current recipe.
Related Feature(s)	F02

UC03: Get Time-Based Recommendation

Use Case ID	UC03
Use Case Name	Get Time-Based Recommendation
Actor(s)	User, LLM Service
Goal	Get recipes that can realistically be completed in the time available.
Preconditions	The recipe database is available.
Trigger	The user enters their available time and clicks Recommend.
Main Success Scenario	1. The user enters the available time. 2. The system performs UC02 Search Recipes («include») to get candidate recipes. 3. The system filters recipes whose preparation plus baking time fits. 4. The agent uses the LLM to rank the recipes and explain the recommendations. 5. The ranked recommendations are displayed.

Alternative / Exception Flows	1. Time is invalid (zero, negative, or not a number): the system asks for a valid time. 3. No recipe fits: the agent suggests the shortest recipes or asks the user to increase the time. 4. LLM Service is unavailable: the system shows an error.
Postconditions	A ranked list of time-feasible recipes with explanations is displayed.
Related Feature(s)	F03

UC04: Check Ingredient Availability

Use Case ID	UC04
Use Case Name	Check Ingredient Availability
Actor(s)	User
Goal	Find out which ingredients the user is missing for a recipe.
Preconditions	A recipe has been selected.
Trigger	The user enters their available ingredients and clicks Check.
Main Success Scenario	1. The user enters or updates the ingredients they have. 2. The system compares them with the recipe's ingredients. 3. The system identifies the missing ingredients. 4. The available and missing ingredients are displayed.
Alternative / Exception Flows	3a. Nothing is missing: the system tells the user they have everything. 3b. Something is missing: the user may request a substitute, which triggers UC05 Find Ingredient Substitution («extend») 1. No ingredients entered: all recipe ingredients are shown as missing.
Postconditions	The user's ingredient list is updated and the availability result is displayed.
Related Feature(s)	F04

UC05: Find Ingredient Substitution

Use Case ID	UC05
Use Case Name	Find Ingredient Substitution

Actor(s)	User, LLM Service
Goal	Get a suitable replacement for a missing ingredient.
Preconditions	A recipe is selected and a missing ingredient has been identified.
Trigger	The user selects a missing ingredient and clicks Find Substitute (extends UC04; used only when something is missing).
Main Success Scenario	1. The user selects the missing ingredient. 2. The agent sends the recipe and ingredient to the LLM. 3. The LLM returns a substitute and how it may affect the recipe. 4. The system displays the substitute and its explanation.
Alternative / Exception Flows	3. No suitable substitute exists: the agent suggests an alternative recipe (via UC02). 2. LLM Service is unavailable: the system shows an error and lets the user retry.
Postconditions	The substitute and its effect on the recipe are displayed.
Related Feature(s)	F05

UC06: Scale Recipe

Use Case ID	UC06
Use Case Name	Scale Recipe
Actor(s)	User
Goal	Adjust a recipe's quantities to the desired servings.
Preconditions	A recipe has been selected.
Trigger	The user enters the desired servings and clicks Scale.
Main Success Scenario	1. The user enters the desired number of servings or items. 2. The system calculates the scaling factor. 3. The system multiplies each ingredient quantity by the factor. 4. The updated recipe is displayed.
Alternative / Exception Flows	1. Value is invalid (zero, negative, or not a number): the system shows an error and asks for a valid number.
Postconditions	The scaled recipe is displayed and the original recipe is unchanged.

Related Feature(s)	F06
---------------------------	-----

UC07: Customize Recipe Taste

Use Case ID	UC07
Use Case Name	Customize Recipe Taste
Actor(s)	User, LLM Service
Goal	Modify a recipe to match a taste preference.
Preconditions	A recipe has been selected.
Trigger	The user enters a taste request (for example, "make it less sweet").
Main Success Scenario	1. The user enters the taste request. 2. The agent sends the recipe and request to the LLM. 3. The LLM proposes adjustments that keep the recipe's overall structure. 4. The system builds the modified recipe. 5. The modified recipe is displayed.
Alternative / Exception Flows	1. Request is unclear: the agent asks the user to clarify. 3. LLM response is unusable or the service is unavailable: the system shows an error and keeps the original recipe.
Postconditions	The modified recipe is displayed and the original is not overwritten.
Related Feature(s)	F07

UC08: Generate Shopping List

Use Case ID	UC08
Use Case Name	Generate Shopping List
Actor(s)	User
Goal	Get a list of the ingredients the user needs to buy.
Preconditions	A recipe has been selected.
Trigger	The user clicks Generate Shopping List.

Main Success Scenario	1. The user clicks Generate Shopping List. 2. The system performs UC04 Check Ingredient Availability («include») to find missing ingredients. 3. The system builds a ShoppingList from the missing items. 4. The shopping list is displayed.
Alternative / Exception Flows	3. Nothing is missing: the system tells the user the list is empty. 2. User has entered no ingredients: all recipe ingredients are added to the list.
Postconditions	A shopping list of missing ingredients is displayed.
Related Feature(s)	F08

UC09: Manage Favorite Recipes

Use Case ID	UC09
Use Case Name	Manage Favorite Recipes
Actor(s)	User
Goal	Save recipes for later and view them again.
Preconditions	A recipe is displayed (to save), or the application is open (to view).
Trigger	The user clicks Favorite on a recipe, or opens the Favorites view.
Main Success Scenario	Save: 1. The user clicks Favorite. 2. The system checks that the recipe is not already saved. 3. The system stores it. 4. A confirmation is shown. View: 1. The user opens Favorites. 2. The system displays the saved recipes.
Alternative / Exception Flows	Save 2. Recipe is already saved: the system tells the user and does not add a duplicate. Save 3. Storage fails: the system shows an error. View 2. There are no favorites: the system shows an empty-list message.
Postconditions	The favorites list is updated (save) or displayed (view).

Related Feature(s)	F09
---------------------------	-----

UC10: Troubleshoot Baking Problem

Use Case ID	UC10
Use Case Name	Troubleshoot Baking Problem
Actor(s)	User, LLM Service
Goal	Get possible causes and fixes for a problem during baking.
Preconditions	A recipe and current step are selected.
Trigger	The user describes a problem and clicks Troubleshoot.
Main Success Scenario	1. The user selects the recipe step and describes the problem. 2. The agent sends the problem, recipe, and step to the LLM. 3. The LLM returns possible causes and solutions. 4. The system displays them.
Alternative / Exception Flows	2a. More information is needed: the agent asks follow-up questions, then resumes at step 2. 2b. LLM Service is unavailable: the system shows an error and lets the user retry.
Postconditions	Possible causes and solutions are displayed.
Related Feature(s)	F10

Task 3 - Feature to Design Traceability

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern
F01	Baking Request: interpret what the user wants to bake	AI	UC01 Submit Baking Request	BakingGUI, AgentController, BakingAgent, ToolManager, BakingRequestTool, LLMClient, BakingRequest	submitRequest(), processRequest(), analyzeRequest(), getTool(), execute(), identifyRequirements(), generateResponse()	SD01	MVC, Facade, Strategy, Factory Method, Adapter
F02	Recipe Search: find recipes matching the requirements	Hybrid	UC02 Search Recipes (included by UC01 and UC03)	BakingAgent, ToolManager, RecipeSearchTool, LLMClient, Recipe	execute(), interpretRequest(), searchRecipes(), generateResponse()	SD01, SD02	Strategy, Factory Method, Adapter
F03	Time Based Recipe Recommendation	Hybrid	UC03 Get Time-Based Recommendation	BakingAgent, ToolManager, RecipeSearchTool, TimeRecommendationTool, LLMClient, Recipe	execute(), filterByTime(), rankRecipes(), rankOptions(), getTotalTime()	SD02	Strategy, Factory Method, Adapter
F04	Ingredient Availability Check	Deterministic	UC04 Check Ingredient Availability	BakingGUI, AgentController, IngredientManager, Recipe, Ingredient	checkIngredients(), getIngredients(), compareIngredients(), findMissingIngredients()	SD03	MVC, Facade
F05	Ingredient Substitution	AI	UC05 Find Ingredient Substitution (extends UC04)	SubstitutionTool, IngredientManager, BakingAgent, ToolManager, LLMClient	execute(), suggestSubstitute(), explainEffect(), findMissingIngredients(), generateResponse()	SD04	Strategy, Factory Method, Adapter
F06	Recipe Scaling	Deterministic	UC06 Scale Recipe	BakingGUI, AgentController, Recipe, Ingredient	scaleRecipe(), updateQuantity()	SD07	MVC, Facade
F07	Taste Customization	AI	UC07 Customize Recipe Taste	TasteCustomizationTool, UserPreferences, Recipe, BakingAgent, LLMClient	execute(), customizeRecipe(), explainChanges(), getPreference(), generateResponse()	SD05	Strategy, Factory Method, Adapter
F08	Shopping List Generation	Deterministic	UC08 Generate Shopping List (includes UC04)	BakingGUI, AgentController, ShoppingList, IngredientManager, Recipe	generateShoppingList(), getIngredients(), compareIngredients(), addItem()	SD03	MVC, Facade
F09	Recipe Favourites	Deterministic	UC09 Manage Favourite Recipes	BakingGUI, AgentController, FavouritesManager, Recipe	addFavourite(), getFavourites(), removeFavourite()	SD08	MVC, Facade

F10	Baking Troubleshooter	AI	UC10 Troubleshoot Baking Problem	TroubleshootingTool, BakingAgent, ToolManager, LLMClient	execute(), analyzeProblem(), suggestSolutions(), requestAdditionalInformation(), generateResponse()	SD06	Strategy, Factory Method, Adapter
-----	-----------------------	----	----------------------------------	--	---	------	-----------------------------------

Task 4 - Explain How Each Feature is Realized

F01: Baking Request

Related Use Case: UC01 Submit Baking Request

Related Sequence Diagram: SD01 Submit Baking Request

Classes involved:

- BakingGUI: collects the request text, servings, available time, ingredients, and dietary restrictions from the user.
- AgentController: the single entry point (Facade) that passes the request to the agent.
- BakingAgent: coordinates the processing, works out the user's intent, and picks the tool.
- ToolManager: returns the tool the agent asks for.
- BakingRequestTool: analyzes the request and extracts its requirements.
- LLMClient: communicates with the LLM.
- BakingRequest: holds the structured request data.

Important methods: BakingGUI.submitRequest(), AgentController.processRequest(), BakingAgent.processRequest(), LLMClient.analyzeRequest(), ToolManager.getTool(), BakingRequestTool.execute(), identifyRequirements(), LLMClient.generateResponse()

Execution: When the user clicks Submit, submitRequest() sends a BakingRequest to AgentController.processRequest(), which passes it to BakingAgent. The agent asks LLMClient.analyzeRequest() for the intent, then gets BakingRequestTool from ToolManager. The tool calls the LLM through generateResponse() and extracts the requirements with identifyRequirements(). The agent then continues with the recipe search and returns the result to the GUI.

Alternative: If the request is unclear, BakingAgent.requestClarification() asks the user a question. If the LLM is unavailable, displayError() is shown.

F02: Recipe Search

Related Use Case: UC02 Search Recipes (included by UC01 and UC03)

Related Sequence Diagram: SD01, SD02

Classes involved:

- BakingAgent: requests the search tool and passes in the requirements.
- ToolManager: provides RecipeSearchTool.
- RecipeSearchTool: interprets the criteria and finds matching recipes.
- LLMClient: helps interpret natural-language criteria.
- Recipe: represents each recipe found.

Important methods: ToolManager.getTool(), RecipeSearchTool.execute(), interpretRequest(), searchRecipes(), LLMClient.generateResponse()

Execution: The agent gets RecipeSearchTool and calls execute() with the requirements. The tool uses the LLM through interpretRequest() to turn them into search criteria, then searchRecipes() looks them up in the recipe repository and returns a List<Recipe>. The GUI shows them with displayRecipe().

Alternative: If nothing matches, the GUI tells the user and lets them change the search.

F03: Time Based Recipe Recommendation

Related Use Case: UC03 Get Time-Based Recommendation

Related Sequence Diagram: SD02 Time Based Recommendation

Classes involved:

- BakingGUI and AgentController: collect the time and forward the request.
- BakingAgent: runs the search, then the time-based tool.
- RecipeSearchTool: provides the candidate recipes (UC02 include).
- TimeRecommendationTool: filters and ranks the recipes.
- LLMClient: ranks the options and explains them.
- Recipe: supplies its total time.

Important methods: submitRequest(), BakingAgent.processRequest(), RecipeSearchTool.searchRecipes(), TimeRecommendationTool.execute(), filterByTime(), rankRecipes(), Recipe.getTotalTime(), LLMClient.rankOptions()

Execution: The user enters their available time. The agent first runs the recipe search to get candidates, then calls TimeRecommendationTool.execute(). The tool calls filterByTime() to keep recipes that fit (using each recipe's getTotalTime()), and rankRecipes() uses LLMClient.rankOptions() to rank and explain the results. The ranked list returns to the GUI.

Alternative: If no recipe fits, the agent suggests the shortest recipes or asks the user to allow more time.

F04: Ingredient Availability Check

Related Use Case: UC04 Check Ingredient Availability

Related Sequence Diagram: SD03 Check Availability and Generate Shopping List

Classes involved:

- BakingGUI: collects the ingredients the user has.
- AgentController: coordinates the check.
- Recipe and Ingredient: supply the required ingredients.
- IngredientManager: compares required and available ingredients.

Important methods: AgentController.checkIngredients(), Recipe.getIngredients(), IngredientManager.compareIngredients(), findMissingIngredients(), BakingGUI.displayResult()

Execution: The GUI calls `checkIngredients(recipe, ingredients)` on the controller. The controller gets the recipe's required ingredients with `getIngredients()` and passes them with the available ones to `IngredientManager.compareIngredients()`. The returned missing ingredients are displayed by `displayResult()`. No LLM is involved.

Alternative: If nothing is missing, the GUI says so. If the user entered no ingredients, every recipe ingredient is reported as missing.

F05: Ingredient Substitution

Related Use Case: UC05 Find Ingredient Substitution (extends UC04)

Related Sequence Diagram: SD04 Find Ingredient Substitution

Classes involved:

- BakingAgent and ToolManager: select and run the tool.
- SubstitutionTool: suggests a replacement and explains its effect.
- IngredientManager: confirms which ingredient is missing.
- LLMClient: generates the suggestion.

Important methods: `BakingAgent.processRequest()`, `ToolManager.getTool()`, `SubstitutionTool.execute()`, `suggestSubstitute()`, `explainEffect()`, `IngredientManager.findMissingIngredients()`, `LLMClient.generateResponse()`

Execution: The user selects a missing ingredient and clicks Find Substitute. The request reaches the agent, which gets SubstitutionTool and calls `execute()`. The tool checks the missing ingredients with IngredientManager, then calls LLMClient through `suggestSubstitute()` for a replacement. `explainEffect()` describes how it changes the recipe, and the GUI displays the result.

Alternative: If no substitute exists, the agent suggests an alternative recipe through a new search.

F06: Recipe Scaling

Related Use Case: UC06 Scale Recipe

Related Sequence Diagram: SD07 Scale Recipe

Classes involved:

- BakingGUI: collects the desired servings.
- AgentController: coordinates the scaling.
- Recipe: scales itself.
- Ingredient: updates its own quantity.

Important methods: `AgentController.scaleRecipe()`, `Recipe.scaleRecipe()`, `Ingredient.updateQuantity()`, `BakingGUI.displayRecipe()`

Execution: The GUI calls `scaleRecipe(recipe, scale)` on the controller. The controller calls `Recipe.scaleRecipe()`, which applies a multiplier to each ingredient through

Ingredient.updateQuantity(). The scaled recipe is returned and shown with displayRecipe(). The calculation is deterministic.

Alternative: If the number is zero, negative, or not a number, displayError() asks for a valid value.

F07: Taste Customization

Related Use Case: UC07 Customize Recipe Taste

Related Sequence Diagram: SD05 Customize Recipe Taste

Classes involved:

- BakingAgent and ToolManager: select and run the tool.
- TasteCustomizationTool: produces the modified recipe.
- UserPreferences: supplies stored taste preferences.
- LLMClient: proposes the adjustments.
- Recipe: the original and modified recipe.

Important methods: BakingAgent.processRequest(), UserPreferences.getPreference(), TasteCustomizationTool.execute(), customizeRecipe(), explainChanges(), LLMClient.generateResponse()

Execution: The user enters a request such as "make it less sweet." The agent reads the stored preferences with getPreference(), then calls TasteCustomizationTool.execute(). The tool asks the LLM for adjustments, builds a modified recipe with customizeRecipe() that keeps the original structure, and explainChanges() summarizes what changed. The GUI displays the modified recipe.

Alternative: If the request is unclear, the agent asks for clarification. If the LLM fails, the original recipe is kept and displayError() is shown.

F08: Shopping List Generation

Related Use Case: UC08 Generate Shopping List (includes UC04)

Related Sequence Diagram: SD03

Classes involved:

- BakingGUI: triggers the request.
- AgentController: coordinates the generation.
- Recipe: supplies the required ingredients.
- IngredientManager: finds the missing ones (the UC04 include).
- ShoppingList: holds the result.

Important methods: AgentController.generateShoppingList(), Recipe.getIngredients(), IngredientManager.compareIngredients(), ShoppingList.addItem(), getItem()

Execution: The user clicks Generate Shopping List. The controller repeats the availability steps from F04 to get the missing ingredients, then adds each one to a ShoppingList with addItem(). The list returns to the GUI and is displayed.

Alternative: If nothing is missing, the GUI reports an empty list. If the user entered no ingredients, all recipe ingredients are listed.

F09: Recipe Favourites

Related Use Case: UC09 Manage Favourite Recipes

Related Sequence Diagram: SD08 Manage Favourite Recipes

Classes involved:

- BakingGUI: provides the Favourite button and Favourites view.
- AgentController: forwards requests.
- FavouritesManager: stores and returns saved recipes.
- Recipe: the saved item.

Important methods: AgentController.addFavourite(), getFavourites(), FavouritesManager.addFavourite(), removeFavourite(), getFavourites()

Execution: When the user clicks Favourite, the controller calls FavouritesManager.addFavourite(recipe), which stores it unless it is already saved, and the GUI shows a confirmation. To view favourites, the controller calls FavouritesManager.getFavourites() and the GUI displays the list. This feature is deterministic.

Alternative: A duplicate is not added and the user is told. An empty list shows an empty message.

F10: Baking Troubleshooter

Related Use Case: UC10 Troubleshoot Baking Problem

Related Sequence Diagram: SD06 Troubleshoot Baking Problem

Classes involved:

- BakingAgent and ToolManager: select and run the tool.
- TroubleshootingTool: analyzes the problem and proposes fixes.
- LLMClient: reasons about the problem.

Important methods: BakingAgent.processRequest(), ToolManager.getTool(), TroubleshootingTool.execute(), analyzeProblem(), suggestSolutions(), requestAdditionalInformation(), LLMClient.generateResponse()

Execution: The user describes a problem and the current step. The agent calls TroubleshootingTool.execute(). The tool sends the description to the LLM, and analyzeProblem() identifies likely causes while suggestSolutions() produces fixes. The GUI displays them.

Alternative: If more detail is needed, `requestAdditionalInformation()` asks the user follow-up questions.